

Chuyên đề: Chặt Nhị Phân Song Song (Parallel Binary Search - PBS)

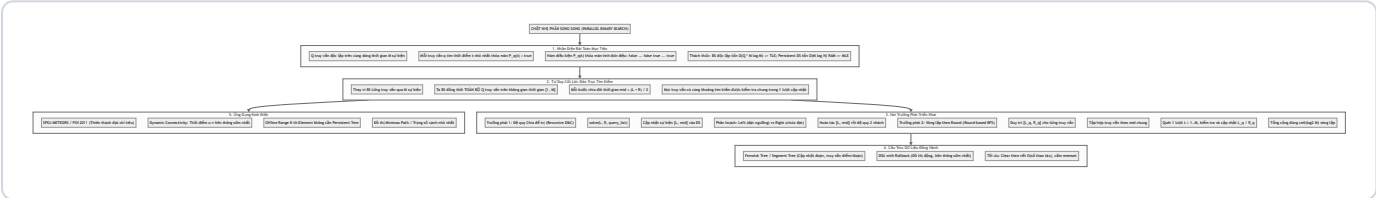
(Parallel Binary Search, Batch Query Processing, Divide & Conquer on Event Timeline, Offline Optimization)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề: Nghịch Lý Hiệu Năng Giữa Bộ Nhớ và Thời Gian

1.1. Mô hình bài toán tổng quát

Trong lập trình thi đấu đỉnh cao (ICPC, OLP, IOI), ta thường xuyên đối mặt với mô hình bài toán sau:

- Cho một hệ thống biến đổi theo thời gian thông qua M sự kiện / thao tác cập nhật $E_1, E_2, \dots, E_M$.
- Có Q truy vấn độc lập dạng: "Đối với đối tượng i , tìm thời điểm $t \in [1, M]$ nhỏ nhất sao cho đối tượng i thỏa mãn một tiêu chuẩn định lượng nào đó (ví dụ: tích lũy đủ K_i tài nguyên, đỉnh u_i và v_i liên thông, thành phần liên thông có kích thước $\geq S_i$)".
- **Tính chất đơn điệu then chốt:** Tiêu chuẩn tích lũy luôn tăng hoặc bảo toàn theo thời gian. Tức là nếu tại thời điểm t , đối tượng đã thỏa mãn điều kiện thì với mọi thời điểm $t' > t$, đối tượng vẫn tiếp tục thỏa mãn ($P_i(t) = \text{true} \implies \forall t' \geq t, P_i(t') = \text{true}$).

1.2. Phân tích các tiếp cận truyền thống & Bế tắc

Giả sử ta cần T_{update} để thêm một sự kiện vào cấu trúc dữ liệu, và T_{query} để kiểm tra một truy vấn. Thông thường $T_{\text{update}}, T_{\text{query}} = O(\log N)$ hoặc $O(1)$.

Hướng tiếp cận	Độ phức tạp thời gian	Độ phức tạp không gian	Nhận xét & Hạn chế
1. Chặt nhị phân độc lập từng truy vấn (Naive BS)	$O(Q \cdot M \log M \cdot T)$	$O(N + M)$	TLE chắc chắn: Với mỗi truy vấn trong Q , ta BS thời điểm <code>mid</code> . Để kiểm tra tại <code>mid</code> , ta phải replay lại <code>mid</code> sự kiện từ đầu. Với $N, M, Q \approx 2 \times 10^5$, tổng phép tính lên tới $2 \cdot 10^5 \times 2 \cdot 10^5 \times 18 \approx 7.2 \times 10^{11} \gg 10^8$.
2. Cấu trúc dữ liệu Bền vững (Persistent Data Structure)	$O((M + Q \log M) \log N)$	$O(M \log N)$	Dễ ăn MLE / Cực khó cài đặt: Dựng Persistent Segment Tree hoặc Persistent Treap qua M thời điểm. Mỗi bước tạo ra $O(\log N)$ nút mới. Với $M = 3 \times 10^5$, bộ nhớ cây ngốn hơn $3 \times 10^5 \times 19 \times 32 \text{ bytes} \approx 180 \text{ MB} - 250 \text{ MB}$. Đặc biệt, nếu cấu trúc dữ liệu là DSU , việc làm persistent DSU cực kỳ cồng kềnh với hằng số thời gian rất lớn $O(\log^2 N)$.
3. Chặt Nhị Phân Song Song (Parallel Binary Search - PBS)	$O((M + Q) \log M \cdot T)$	$O(N + M + Q)$	Hoàn hảo tối ưu: Không cần lưu lại lịch sử quá khứ! Chỉ cần một cấu trúc dữ liệu tĩnh chuẩn duy nhất (Fenwick Tree, DSU with Rollback). Bộ nhớ phẳng chỉ vài Megabytes, thời gian chạy cực nhanh.

Xuất xứ lịch sử: Thuật toán Chặt Nhị Phân Song Song được phát minh và chuẩn hóa bởi chuyên gia thuật toán **Michal Forišek** và trở nên nổi tiếng toàn cầu sau kỳ thi **Polish Olympiad in Informatics (POI 2011)** với bài toán huyền thoại **Meteors (Meteory)**.

2. Bản Chất Toán Học & Cơ Chế Hoạt Động Của PBS

2.1. Đơn điệu hóa không gian nghiệm

Gọi $f_q(t)$ là giá trị tích lũy của truy vấn q sau khi đã kích hoạt toàn bộ các sự kiện $E_1, E_2, \dots, E_t$.

Do các sự kiện mang tính đóng góp không âm (hoặc liên thông không giảm):

$$\forall t_1 \leq t_2 \implies f_q(t_1) \leq f_q(t_2)$$

Mục tiêu là tìm:

$$ans_q = \min\{t \in [1, M] \mid f_q(t) \geq K_q\}$$

Nếu xem việc kiểm tra hàm $f_q(\text{mid}) \geq K_q$ như một bộ lọc nhị phân:

- Bất kỳ truy vấn nào thỏa mãn tại `mid` thì nghiệm tối ưu của nó chắc chắn nằm trong nửa khoảng bên trái: $[L_q, \text{mid}]$.
- Bất kỳ truy vấn nào chưa thỏa mãn tại `mid` thì nghiệm tối ưu của nó chắc chắn nằm trong nửa khoảng bên phải: $[\text{mid} + 1, R_q]$.

2.2. Ý tưởng cách mạng: Gộp chung bước kiểm tra

Thay vì chạy cấu trúc dữ liệu riêng cho từng truy vấn, PBS **gộp tất cả các truy vấn có cùng khoảng tìm kiếm $[L, R]$ lại thành một danh sách chung.**

Khi chia đôi tại $\text{mid} = \lfloor (L + R)/2 \rfloor$:

1. Ta đưa các sự kiện từ L đến mid vào cấu trúc dữ liệu **đúng 1 lần**.
 2. Duyệt qua toàn bộ danh sách truy vấn: kiểm tra truy vấn nào đạt ngưỡng, truy vấn nào chưa đạt.
 3. Chia danh sách truy vấn thành 2 danh sách con:
 - Q_{left} : Gồm các truy vấn đã đạt ngưỡng $\rightarrow$ nghiệm thuộc $[L, \text{mid}]$.
 - Q_{right} : Gồm các truy vấn chưa đạt ngưỡng $\rightarrow$ nghiệm thuộc $[\text{mid} + 1, R]$.
 4. Hoàn tác cấu trúc dữ liệu về trạng thái trước đó (Rollback / Clear) và đệ quy xuống 2 nhánh con.
-

3. Hai Trường Phái Triển Khai: Đệ Quy (D&C) vs Vòng Lặp (Round-based)

Trong thực tế thi đấu, có hai trường phái hiện thực PBS: **Chia để trị đệ quy** và **Vòng lặp BFS theo Round**. Mỗi phương pháp có những ưu và nhược điểm riêng.

3.1. Trường phái 1: Chia để trị đệ quy (Recursive Divide & Conquer)

Đây là cách tiếp cận phổ biến nhất, mã nguồn rất thanh thoát và mô phạm.

```

Hàm solve(L, R, QueryList):
    Nếu L == R:
        Với mọi q in QueryList:
            ans[q] = L
        Trả về

    mid = (L + R) / 2

    // Bước 1: Áp dụng các sự kiện trong nửa trái [L, mid] vào cấu trúc dữ liệu
    Cho t chạy từ L đến mid:
        Áp dụng sự kiện E[t] vào Data Structure

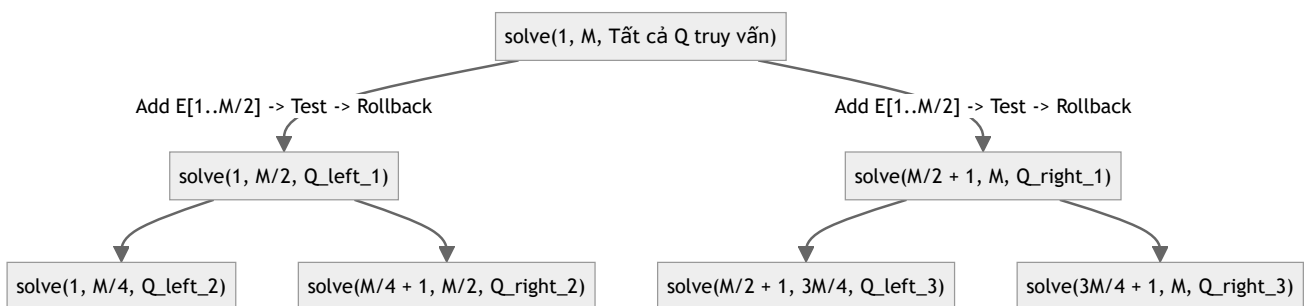
    // Bước 2: Kiểm tra từng truy vấn trong QueryList
    Khởi tạo LeftList = [], RightList = []
    Với mỗi q in QueryList:
        current_val = truy vấn giá trị tích lũy của q từ Data Structure
        Nếu current_val >= K[q]:
            LeftList.push_back(q)
        Ngược lại:
            // Có 2 biến thể:
            // Biến thể A: giữ nguyên K[q], khi sang nhánh phải ta phải replay lại [L, mid]
            // Biến thể B (Khuyến dùng): K[q] -= current_val, nhánh phải chỉ cần xét đóng góp mới từ
            RightList.push_back(q)

    // Bước 3: Hoàn tác các sự kiện từ L đến mid để trả lại trạng thái sạch cho cấu trúc dữ liệu
    Cho t chạy từ L đến mid:
        Hoàn tác sự kiện E[t] khỏi Data Structure

    // Bước 4: Đệ quy giải quyết độc lập 2 nhánh
    solve(L, mid, LeftList)
    solve(mid + 1, R, RightList)

```

Sơ đồ phân nhánh cây đệ quy:



💡 Mẹo / Tối ưu:

****Kỹ thuật Biến thể B (Trừ hao ngưỡng cần đạt):****

Nếu bài toán cho phép tính chất giao hoán và cộng dồn (như thêm phần tử vào tổng), khi chuyển q sang `RightList`, ta có thể trừ trực tiếp giá trị đóng góp của đoạn $[L, mid]$ vào K_q : $K_q \leftarrow K_q - \text{val}_{[L, mid]}$.

Nhờ đó, khi nhánh phải `solve(mid + 1, R, RightList)` chạy, nó **không** cần phải nạp lại các sự kiện từ `1 . . . mid`! Cấu trúc dữ liệu hoàn toàn bắt đầu từ trạng thái rỗng trên đoạn thời gian mới $[mid + 1, R]$.

3.2. Trường phái 2: Vòng lặp BFS theo Round (Round-based BFS)

Phương pháp này không dùng đệ quy mà sử dụng một vòng lặp ngoài gồm đúng $\lceil \log_2(M + 1) \rceil$ lượt (rounds).

Cơ chế hoạt động:

1. Ban đầu, mọi truy vấn i đều có khoảng tìm kiếm nghiệm $L_i = 1, R_i = M + 1$.
2. Trong mỗi round:
3. Kiểm tra xem còn truy vấn nào chưa hội tụ ($L_i < R_i$) hay không. Nếu không còn, dừng thuật toán.
4. Với các truy vấn còn đang tìm kiếm, tính $mid_i = \lfloor (L_i + R_i) / 2 \rfloor$.
5. Gom các truy vấn có cùng mid_i vào danh sách chờ tại thời điểm đó: `buckets[mid_i].push_back(i)`.
6. **Quét 1 lượt duy nhất từ thời điểm $t = 1 \dots M$:**
 - Áp dụng sự kiện E_t vào cấu trúc dữ liệu.
 - Kiểm tra toàn bộ các truy vấn nằm trong `buckets[t]`.
 - Nếu truy vấn i thỏa mãn: thu hẹp $R_i \leftarrow t$.
 - Ngược lại: thu hẹp $L_i \leftarrow t + 1$.
7. Xóa sạch (Clear) cấu trúc dữ liệu một lần duy nhất vào cuối round.

```

// Minh họa khung code Round-based PBS
std::vector<int> L(Q, 1), R(Q, M + 1);
bool changed = true;

while (changed) {
    changed = false;
    std::vector<std::vector<int>> mid_buckets(M + 1);

    for (int i = 0; i < Q; ++i) {
        if (L[i] < R[i]) {
            int mid = L[i] + (R[i] - L[i]) / 2;
            mid_buckets[mid].push_back(i);
            changed = true;
        }
    }
    if (!changed) break;

    data_structure.clear();
    for (int t = 1; t <= M; ++t) {
        data_structure.apply_event(events[t]);
        for (int q_id : mid_buckets[t]) {
            if (data_structure.check_condition(queries[q_id])) {
                R[q_id] = t;      // Nghiệm nằm ở t hoặc trước t
            } else {
                L[q_id] = t + 1;  // Nghiệm bắt buộc phải sau t
            }
        }
    }
}

```

3.3. Bảng so sánh toàn diện giữa 2 Trường phái

Tiêu chí so sánh	Trường phái 1: Chia để trị (Recursive D&C)	Trường phái 2: Vòng lặp Round-based (BFS)
Số lần cập nhật sự kiện	Mỗi sự kiện được thêm vào và hoàn tác ở các tầng cây con. Tổng số phép thêm qua toàn cây là $O(M \log M)$.	Đúng M sự kiện được nạp vào mỗi round. Tổng cộng $M \lceil \log_2 M \rceil$ phép thêm.
Cơ chế hoàn tác (Rollback)	Bắt buộc phải có cơ chế hoàn tác từng phần (undo/rollback) hoặc trừ hao chỉ tiêu.	Không cần Rollback! Chỉ cần hàm <code>clear()</code> toàn bộ cấu trúc dữ liệu sau mỗi round. Cực kỳ tiện lợi cho các cấu trúc khó rollback.
Bộ nhớ phụ (Auxiliary Space)	Cần vector chứa id truy vấn qua các hàm đệ quy ($O(Q \log M)$ nếu cấp phát mới, hoặc $O(Q)$ nếu dùng kỹ thuật 2 con trỏ / in-place partition).	Cần mảng <code>L</code> , <code>R</code> và mảng các buckets gom nhóm: $O(M + Q)$. Cực kỳ tiết kiệm bộ nhớ.
Độ thân thiện khi code	Tư duy toán học rất tự nhiên, ngắn gọn khi dùng chung với Fenwick Tree.	Cấu trúc code dạng phẳng, không sợ tràn stack đệ quy (Stack Overflow), debug từng round rất dễ dàng.

4. Chứng Minh Độ Phức Tạp Thuật Toán

4.1. Độ phức tạp thời gian (Time Complexity)

Xét cây đệ quy của phương pháp Chia để trị (hoặc các round của phương pháp BFS):

1. Độ sâu của cây đệ quy:

Không gian thời gian ban đầu có độ dài M . Mỗi bước chia đôi khoảng tìm kiếm:

$$\text{Độ sâu} = \lceil \log_2 M \rceil$$

2. Khối lượng công việc tại một tầng (Level) của cây:

- **Xử lý truy vấn:** Mỗi truy vấn $q \in \{1, \dots, Q\}$ xuất hiện trong **đúng một nút** ở mỗi tầng đệ quy (do truy vấn được phân hoạch về nhánh trái hoặc nhánh phải, không bị nhân bản). Do đó, tổng số lượt kiểm tra truy vấn trên toàn bộ một tầng là:

$$\sum_{\text{các nút ở tầng } k} |\text{QueryList}| = Q$$

Chi phí kiểm tra tại tầng k là $O(Q \cdot T_{\text{query}})$.

- **Xử lý sự kiện:** Tại mỗi nút đệ quy quản lý đoạn thời gian $[L, R]$, ta duyệt qua đoạn $[L, \text{mid}]$ có độ dài $\frac{R-L+1}{2}$.

Tổng độ dài các đoạn $[L, \text{mid}]$ của tất cả các nút cùng thuộc tầng k không bao giờ vượt quá M :

$$\sum_{\text{các nút ở tầng } k} (\text{mid} - L + 1) \leq M$$

Chi phí cập nhật và hoàn tác tại tầng k là $O(M \cdot T_{\text{update}})$.

3. Tổng chi phí toàn bộ thuật toán:

Lấy tổng qua tất cả $\lceil \log_2 M \rceil$ tầng:

$$T_{\text{total}} = \sum_{k=1}^{\lceil \log_2 M \rceil} O((M \cdot T_{\text{update}} + Q \cdot T_{\text{query}})) = O((M \cdot T_{\text{update}} + Q \cdot T_{\text{query}}) \log M)$$

Đánh giá thực tế:

Với $N, M, Q = 2 \times 10^5$, cấu trúc Fenwick Tree có $T_{\text{update}} = T_{\text{query}} = O(\log N)$.

Tổng thời gian thực thi:

$$O((M + Q) \log M \log N) \approx 4 \times 10^5 \times 18 \times 18 \approx 1.29 \times 10^8 \text{ phép tính}$$

Chạy hoàn toàn êm ái trong khoảng **0.4s - 0.7s** trên các máy chấm hiện đại (Codeforces, CSES, VNOJ).

4.2. Độ phức tạp không gian (Space Complexity)

- Cấu trúc dữ liệu chính (Fenwick Tree, DSU): $O(N)$.
- Lưu trữ sự kiện và thông tin truy vấn gốc: $O(M + Q)$.
- Nếu cài đặt đệ quy truyền thống với `vector` cục bộ: tại mỗi tầng đệ quy, tổng kích thước các vector là Q . Độ sâu cây là $\log M \implies O(Q \log M)$.
- Tối ưu In-place (Dùng mảng hoán vị):** Ta có thể duy trì một mảng chỉ số toàn cục `p[1..Q]` và dùng kỹ thuật phân hoạch hai con trỏ (hoặc `std::stable_partition`) để gom các truy vấn thỏa mãn về đầu và chưa thỏa mãn về cuối đoạn. Khi đó bộ nhớ phụ đưa về $O(N + M + Q)$ tuyệt đối!

5. Các Dạng Bài Toán Kinh Điển & Phân Tích Kỹ Thuật

5.1. Dạng 1: Bài toán Mưa Thiên Thạch (Meteors - POI 2011 / SPOJ METEORS)

Mô tả bài toán:

- Quỹ đạo gồm M trạm vũ trụ xếp thành vòng tròn $1, 2, \dots, M$ (với trạm M nối tiếp trạm 1).
- Có N quốc gia, mỗi quốc gia sở hữu một tập hợp các trạm vũ trụ xác định. Quốc gia i cần thu thập tối thiểu P_i đơn vị mẫu thiên thạch.
- Có K đợt mưa thiên thạch theo thời gian. Đợt thứ t rơi xuống cung tròn từ trạm L_t đến trạm R_t (nếu $L_t > R_t$ thì rơi từ $L_t \dots M$ và $1 \dots R_t$), mỗi trạm trong vùng nhận thêm A_t đơn vị thiên thạch.
- Yêu cầu:** Với mỗi quốc gia i , xác định đợt mưa thiên thạch sớm nhất mà quốc gia đó đạt chỉ tiêu P_i . Nếu sau cả K đợt vẫn không đủ, in ra `"NIE"`.

Mô hình hóa & Cấu trúc dữ liệu:

- Cấu trúc dữ liệu: **Fenwick Tree (Cây chỉ số nhị phân)** hỗ trợ thao tác Range Add và Point Query (hoặc mảng hiệu sai Difference Array trên BIT).
- Cung tròn: Tách thành tối đa 2 đoạn $[L_t, M]$ và $[1, R_t]$.
- Để kiểm tra quốc gia i : Lấy tổng thiên thạch tích lũy tại tất cả các trạm thuộc sở hữu của quốc gia i .

Tránh cạn kiệt tràn số (Saturating Arithmetic):

Một quốc gia có thể có hàng chục ngàn trạm, mỗi trạm nhận 10^9 thiên thạch qua hàng ngàn đợt mưa $\implies$ tổng tích lũy có thể vượt quá $2^{64} - 1$ gây tràn số `unsigned long long`.

Giải pháp: Áp dụng phép cộng chặn tràn:

```
unsigned long long sum = 0;
for (int station : stations_of_country[i]) {
    sum += bit.query(station);
    if (sum >= req[i]) { // Chặn tràn ngay khi đạt chỉ tiêu
        sum = req[i];
        break;
    }
}
```

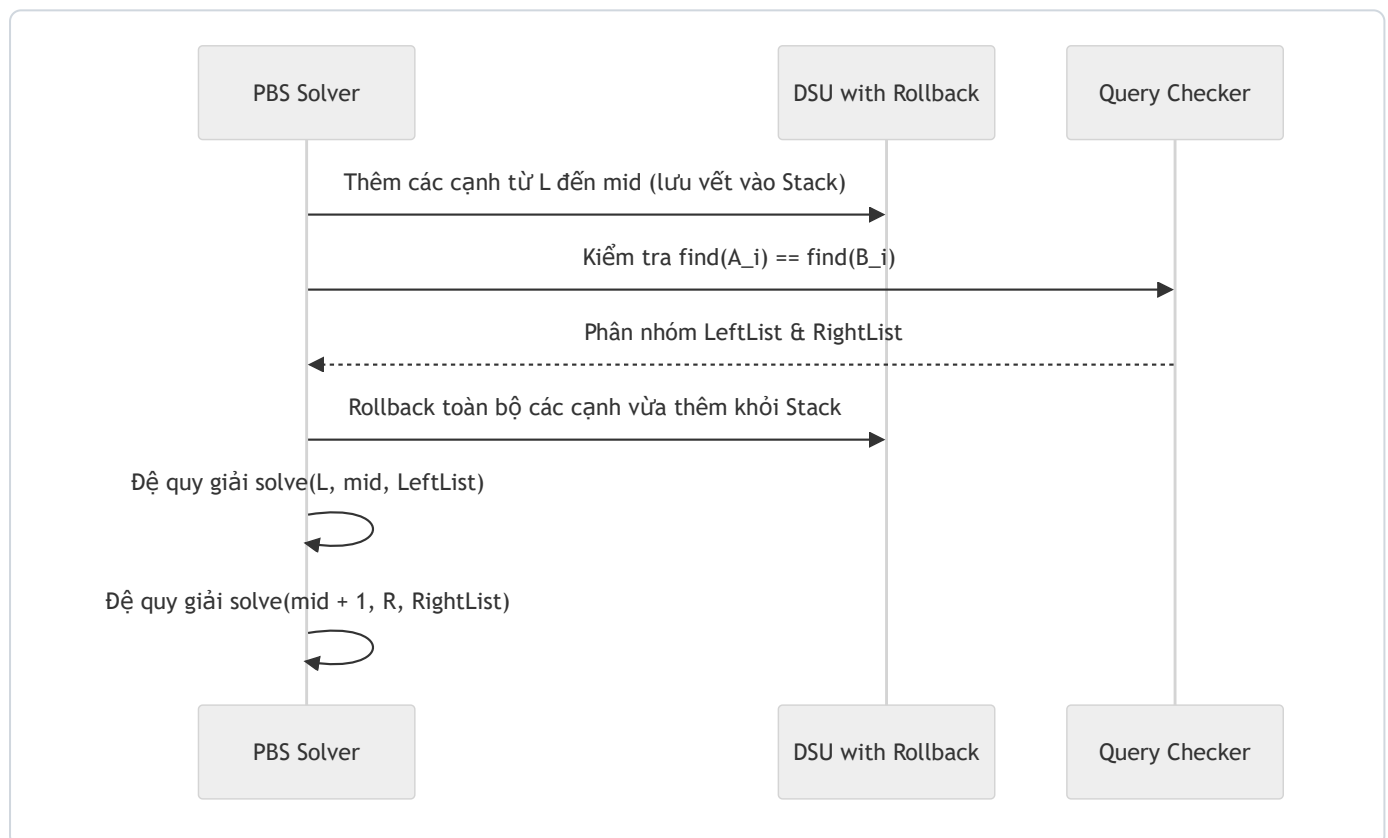
5.2. Dạng 2: Thời Điểm Liên Thông Sớm Nhất (Earliest Connectivity in Dynamic Graph)

Mô tả bài toán:

- Cho đồ thị vô hướng ban đầu rỗng gồm N đỉnh.
- Lần lượt thêm M cạnh (u_t, v_t) tại các thời điểm $t = 1, 2, \dots, M$.
- Có Q truy vấn: Tìm thời điểm sớm nhất để đỉnh A_i và B_i thuộc cùng một thành phần liên thông.

Tiếp cận bằng PBS:

- Hàm điều kiện: Tại thời điểm mid , hai đỉnh A_i và B_i đã liên thông chưa?
- Cấu trúc dữ liệu: **Disjoint Set Union có khả năng hoàn tác (DSU with Rollback)**.
- Chú ý: DSU with Rollback sử dụng tối ưu hóa kích thước cây (Union by Rank/Size) nhưng **KHÔNG ĐƯỢC nén đường đi (Path Compression)** để đảm bảo mỗi thao tác gộp chỉ thay đổi $O(1)$ biến, có thể lưu vào ngăn xếp và hoàn tác trong $O(1)$.
- Độ phức tạp: $O((M + Q) \log M \log N)$.



5.3. Dạng 3: Tìm Phần Tử Nhỏ Thứ K Trên Đoạn (Offline Range K-th Smallest)

Bài toán:

Cho mảng A gồm N phần tử. Có Q truy vấn: tìm phần tử nhỏ thứ K trong đoạn con $A[L \dots R]$.

Tại sao dùng PBS khi đã có Persistent Segment Tree?

- Persistent Segment Tree tốn $O(N \log N)$ nút bộ nhớ. Trong các kỳ thi với giới hạn bộ nhớ cực hẹp (ví dụ: 32 MB — 64 MB), Persistent Tree sẽ bị Memory Limit Exceeded.
- PBS thực hiện chặt nhị phân song song trên **miền giá trị** $[V_{\min}, V_{\max}]$:
- Tại bước **mid**, ta đưa các phần tử có giá trị $\leq \text{mid}$ vào Fenwick Tree (đặt vị trí của chúng thành 1).
- Truy vấn $[L, R]$ đếm xem có bao nhiêu phần tử trong đoạn có giá trị $\leq \text{mid}$ bằng thao tác `bit.query(L, R)`.
- Nếu số lượng $\geq K$: đáp án nằm trong khoảng giá trị nhỏ hơn $[V_{\min}, \text{mid}]$.
- Ngược lại: đáp án nằm trong khoảng lớn hơn $[\text{mid} + 1, V_{\max}]$ và cập nhật $K \leftarrow K - \text{count}$.
- Bộ nhớ của PBS chỉ tốn một cây Fenwick Tree kích thước N : **dưới 2 MB RAM!**

6. Cạm Bẫy Phòng Thi & Kỹ Thuật Tối Ưu Tột Đỉnh

6.1. Cạm bẫy "Xóa sạch bộ nhớ bằng memset" (The Memset Trap)

Trong hàm đệ quy `solve(L, R, QueryList)`:

Nhiều thí sinh có thói quen gọi `memset(bit, 0, sizeof(bit))` để reset cấu trúc dữ liệu.

- Kích thước của mảng là N .
- Cây đệ quy có $2^{\log M} = M$ lá.
- Việc gọi `memset` tại mỗi nút khiến chi phí phụ tốn tới $O(M \cdot N)$!
- Với $N, M = 2 \times 10^5$, $M \cdot N = 4 \times 10^{10} \implies$ **TLE tức thì!**

Lưu ý đặc biệt:

****Quy tắc Vàng:**** Chỉ hoàn tác (clear/undo) ****đúng những vị trí đã bị thay đổi****.

Nếu đã thêm sự kiện từ $t = L \dots \text{mid}$, hãy chạy một vòng lặp hoàn tác $t = L \dots \text{mid}$ với giá trị nghịch đảo (ví dụ: ``add(pos, -val)``). Chi phí hoàn tác sẽ tương đương chi phí cập nhật, giữ nguyên tính tối ưu $O((M + Q) \log M)$.

6.2. Xử lý truy vấn vô nghiệm (Unsatisfiable Queries)

Luôn đặt cận phải của không gian tìm kiếm là $M + 1$.

- Bất kỳ truy vấn nào khi kết thúc thuật toán có nghiệm là $M + 1$ đồng nghĩa với việc dù đã kích hoạt toàn bộ M sự kiện, truy vấn đó vẫn không bao giờ thỏa mãn điều kiện.
- Điều này giúp code không cần phải viết hàm kiểm tra đặc biệt riêng cho các trường hợp vô nghiệm (như in ra `-1` hoặc `"NIE"`).

6.3. Tối ưu In-Place Partitioning (Tránh phân mảnh bộ nhớ RAM)

Thay vì cấp phát `vector<int> left_list, right_list` ở mỗi tầng đệ quy gây phân mảnh bộ nhớ và tốn chi phí copy:

Ta duy trì một mảng toàn cục `query_ids[1..Q]`.

Tại mỗi nút quản lý đoạn chỉ số `[ql, qr]`, ta dùng mảng tạm `temp_ids` để phân loại:

```

int left_cnt = 0, right_cnt = 0;
for (int i = ql; i <= qr; ++i) {
    int q_id = query_ids[i];
    if (check_satisfied(q_id)) {
        temp_left[left_cnt++] = q_id;
    } else {
        temp_right[right_cnt++] = q_id;
    }
}
// Chép đè lại vào query_ids theo thứ tự trái rồi phải
for (int i = 0; i < left_cnt; ++i) query_ids[ql + i] = temp_left[i];
for (int i = 0; i < right_cnt; ++i) query_ids[ql + left_cnt + i] = temp_right[i];

int split_point = ql + left_cnt - 1;
solve(L, mid, ql, split_point);
solve(mid + 1, R, split_point + 1, qr);

```

Cách cài đặt này đẩy hiệu năng lên mức tối đa, bộ nhớ cache CPU L1/L2 hoạt động hoàn hảo và loại bỏ hoàn toàn chi phí cấp phát vùng nhớ động (`malloc` / `free`).

7. Bảng Tổng Hợp So Sánh Các Kỹ Thuật Nâng Cao

Kỹ thuật	Trọng tâm giải quyết	Khả năng Online	Bộ nhớ tiêu thụ	Độ phức tạp thời gian	Hạn chế lớn nhất
Parallel Binary Search	Tìm mốc thời gian thỏa mãn sớm nhất	✗ Ngoại tuyến	$O(N + M + Q)$ (Cực nhỏ)	$O((M + Q) \log M \log N)$	Bắt buộc phải biết trước toàn bộ truy vấn; điều kiện phải đơn điệu
Persistent Segment Tree	Truy vấn lịch sử quá khứ tại thời điểm t bất kỳ	✓ Trực tuyến	$O(M \log N)$ (Rất lớn)	$O((M + Q) \log N)$	Tiêu tốn nhiều RAM; cực khó áp dụng cho đồ thị (DSU)
Thuật toán Mo	Truy vấn hàm mục tiêu trên đoạn $[L, R]$ bất kỳ	✗ Ngoại tuyến	$O(N + Q)$	$O((N + Q)\sqrt{N})$	Không hỗ trợ cập nhật dữ liệu; thời gian phụ thuộc căn bậc 2
Time Divide & Conquer	Duy trì đồ thị với cạnh có khoảng sống $[t_{in}, t_{out}]$	✗ Ngoại tuyến	$O(Q \log Q)$	$O(Q \log^2 Q)$	Đòi hỏi cấu trúc dữ liệu hỗ trợ Rollback

8. Tài liệu Tham khảo & Đọc thêm

1. VNOI Wiki:

2. [Chặt nhị phân song song \(Parallel Binary Search\)](#) — Phân tích chi tiết bài toán POI Meteors.
 3. **Codeforces Community:**
 4. [Parallel Binary Search Tutorial](#) — Kỹ thuật đảo trực tìm kiếm đa truy vấn trên dòng sự kiện.
 5. **Algorithms Thread (William Lin / Neal Wu):**
 6. [Episode 3: Parallel Binary Search](#) — Hướng dẫn cài đặt in-place partition tối ưu bộ nhớ.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.